



PROG IV:

Testes, Git e Boas Práticas

Profª. Tainá Isabela



Controle de Versão Distribuído

Antes da consolidação dos sistemas distribuídos, o gerenciamento de histórico de código sofria com pontos únicos de falha e severas restrições de concorrência em servidores centralizados. O advento do Git transformou esse cenário ao permitir que cada **desenvolvedor possua uma cópia integral e local de todo o repositório, incluindo o histórico completo de modificações.**

Esse modelo descentralizado confere **autonomia às equipes, viabiliza operações rápidas sem dependência de conexão de rede** constante e fornece mecanismos robustos para **ramificação e fusão de linhas de desenvolvimento**. O versionamento deixa de ser um mero backup para se tornar uma ferramenta ativa de colaboração e **auditoria de software.**

Commit e as Áreas de Estados do Git

No funcionamento interno do Git, um commit não funciona como uma gravação incremental de diferenças textuais, mas como um instantâneo fotográfico, ou snapshot, do **estado dos arquivos naquele exato momento**. Para que um arquivo seja registrado nesse histórico, ele transita por **três estados lógicos** bem definidos dentro do diretório de trabalho local.

O primeiro estado é o **Working Directory**, onde as modificações são feitas livremente pelo programador. Em seguida, os arquivos selecionados são movidos para a **Staging Area** ou indexador por meio do comando `git add`. Somente após essa triagem o comando `git commit` consolida os dados na base permanente do repositório local.

Boas Práticas de Commits e Rastreabilidade

O histórico de commits de um projeto deve funcionar como uma **documentação cronológica compreensível** sobre a evolução da arquitetura do software. A adoção de **convenções textuais**, como o modelo Conventional Commits, **padroniza as mensagens utilizando prefixos específicos** para cada natureza de alteração, a exemplo de **feat**: para novas funcionalidades e **fix**: para correção de bugs.

As mensagens devem **descrever claramente o motivo da alteração** de forma impessoal e imperativa, evitando termos vagos como "código atualizado" ou "ajustes". Commits pequenos e **atômicos** facilitam a localização de erros, simplificam revisões de código e tornam processos de reversão menos traumáticos.

Novo recurso	✨ :sparkles:
Lista de ideias (tasks)	📅 :soon:
Bugfix	🐛 :bug:
Documentação	📖 :books:
Testes	🧪 :test_tube:
Adicionando um teste	✅ :white_check_mark:
Teste de aprovação	✔️ :heavy_check_mark:
Acessibilidade	♿️ :wheelchair:
Texto	🖋️ :pencil:
Package.json em JS	📦 :package:
Em progresso	🚧 :construction:
Arquivos de configuração	🔧 :wrench:
Removendo uma dependência	➖ :heavy_minus_sign:
Adicionando uma dependência	➕ :heavy_plus_sign:
Revertendo mudanças	💥 :boom:
Alterações de revisão de código	🔥 :ok_hand:
Refatoração	♻️ :recycle:
Mover/Renomear	🚚 :truck:

✨ feat: adiciona link em ícones

 geovanaborba authored on Dec 7, 2023

✨ feat: adiciona links emoji

 geovanaborba authored on Dec 7, 2023

✨ feat: adiciona links emoji

 geovanaborba authored on Dec 7, 2023

Commits on Nov 13, 2023

🐛 fix: Arrumando bugs nas tabelas

 iuricode authored on Nov 13, 2023

📖 docs: Revisão dos links no readme

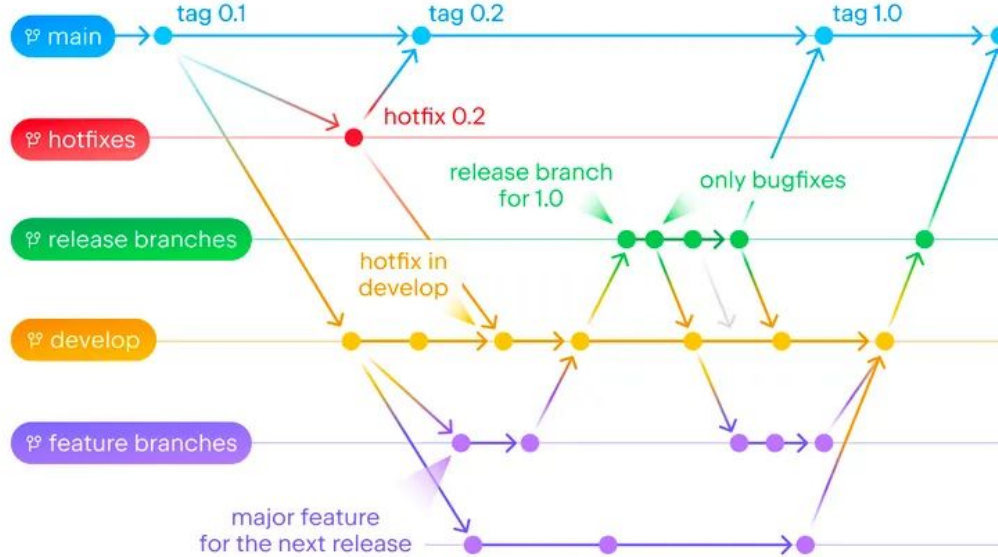
 iuricode authored on Nov 13, 2023

Git Flow e Trunk Based Development

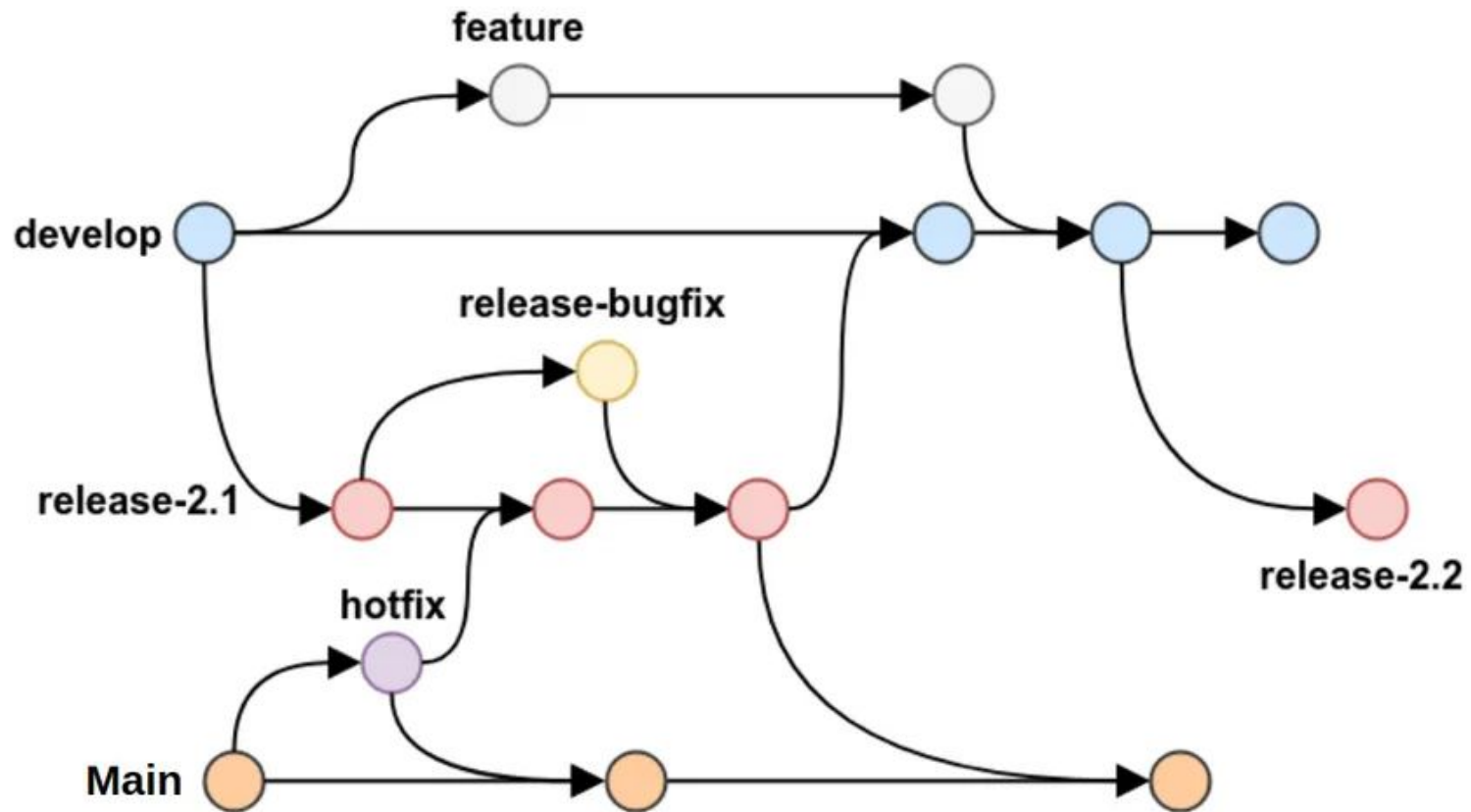
O gerenciamento de equipes de desenvolvimento concorrentes exige uma metodologia para organizar o isolamento e a integração contínua do código. O modelo tradicional **Git Flow** propõe uma estrutura rígida de ramificações de longa duração, divididas em **ramos principais** como main para **produção**, **develop** para **integração**, além de **ramificações temporárias para recursos específicos e correções urgentes**.

Em contrapartida, abordagens modernas como o **Trunk Based Development** priorizam integrações frequentes diretamente no ramo principal, reduzindo drasticamente a ocorrência de conflitos complexos de fusão de arquivos conhecidos como **merge hells** e **acelerando o ritmo de entrega do software**.

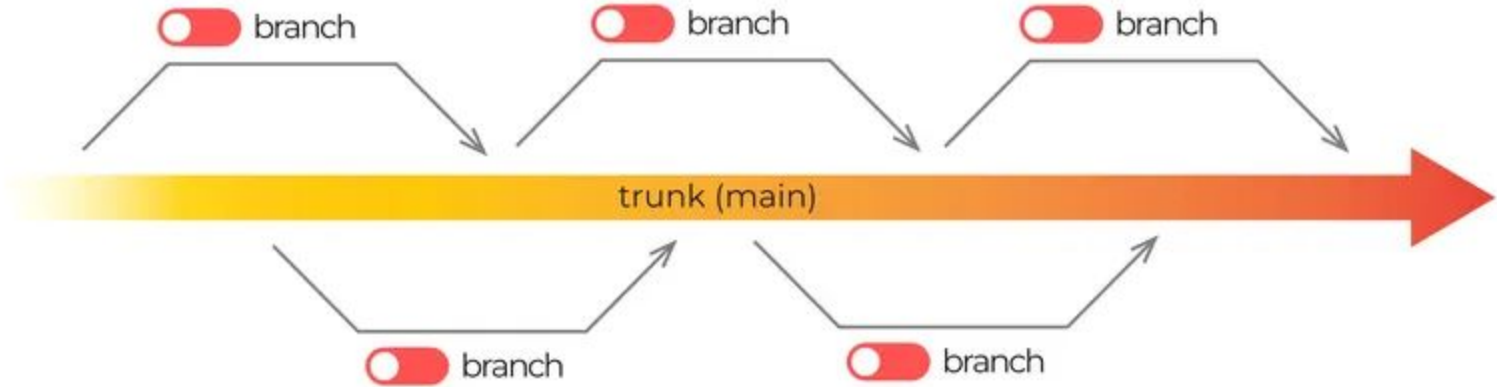
Git flow



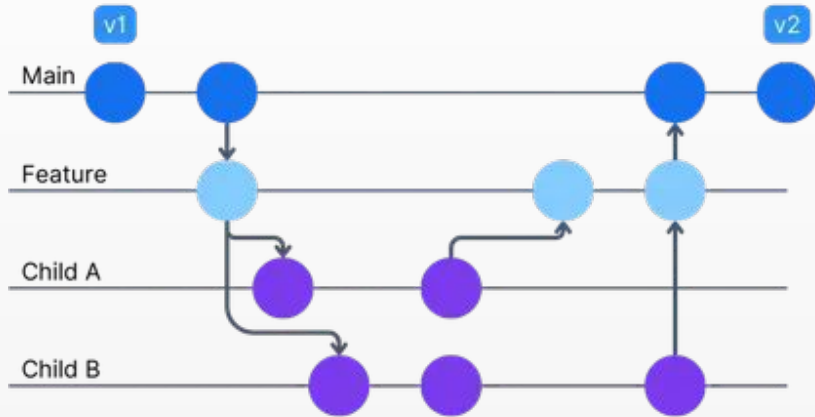
- The **main** branch is for production code only.
- The **develop** branch is for development code.
- **feature** branches are created from the **develop** branch.
- **hotfix** branches are created from the **main** branch.
- **release** branches are created from the **develop** branch.



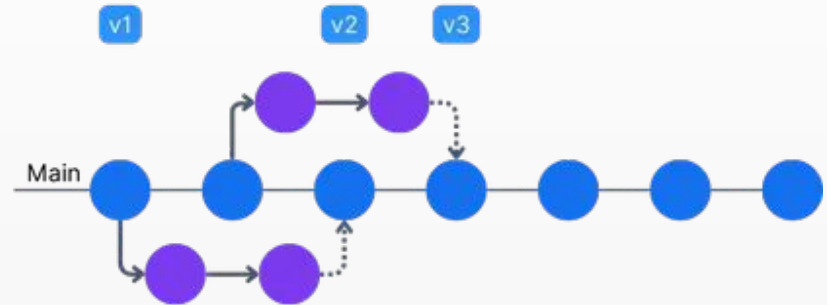
Trunk-Based Development with Feature Flags



Feature-Based Development



Trunk-Based Development



Fundamentos de Testes Unitários no Backend

Os testes unitários ocupam a **base da pirâmide de testes de software** e têm como objetivo validar de forma isolada a menor unidade avaliável de um sistema, geralmente um método ou função de uma classe de backend. Esse tipo de teste foca exclusivamente na **lógica interna da unidade**, operando na memória do computador para garantir velocidades de execução extremamente rápidas.

Ao isolar a unidade, o teste verifica se, para um **determinado conjunto de dados de entrada**, a **função retorna exatamente a saída esperada** ou dispara as exceções corretas. Esse mecanismo fornece feedback instantâneo ao desenvolvedor durante a escrita do código.

Fundamentos de Testes Unitários no Backend

Os testes unitários ocupam a **base da pirâmide de testes de software** e têm como objetivo validar de forma isolada a menor unidade avaliável de um sistema, geralmente um método ou função de uma classe de backend. Esse tipo de teste foca exclusivamente na **lógica interna da unidade**, operando na memória do computador para garantir velocidades de execução extremamente rápidas.

Ao isolar a unidade, o teste verifica se, para um **determinado conjunto de dados de entrada**, a **função retorna exatamente a saída esperada** ou dispara as exceções corretas. Esse mecanismo fornece feedback instantâneo ao desenvolvedor durante a escrita do código.

Dublês de Testes

Funções de backend raramente operam de forma isolada, interagindo constantemente com conexões de banco de dados, chamadas de rede e serviços externos de arquivos. Para manter a natureza isolada e determinística do teste unitário, essas dependências reais devem ser substituídas por dublês de teste, comumente categorizados como **Mocks, Stubs ou Fakes**.

Um objeto do tipo Stub fornece respostas predefinidas para as chamadas feitas pela unidade sob teste, enquanto um Mock foca na verificação do comportamento, atestando se determinado método externo foi invocado com os parâmetros corretos, sem executar a lógica real da dependência.

PASS test/user.controller.spec.ts

UserController Tests

- ✓ Should be defined (1 ms)
- ✓ Should get user
- ✓ Should get users
- ✓ Should add user
- ✓ Should update user
- ✓ Should delete user (1 ms)

Test Suites: 1 passed, 1 total

Tests: 6 passed, 6 total

Snapshots: 0 total

Time: 1.223 s, estimated 2 s

Ran all test suites.

```

TS user.entity.ts  TS user.service.ts  TS user.module.ts  TS user.service.spec.ts X  TS user.controller.ts  TS user.dto.ts

src > features > user > v1 > tests > TS user.service.spec.ts > describe('UserService') callback > test('CreateUser_ShowdBeTrue') callback

7
8 describe('UserService', () => {
9   let service: UserService;
10  let userRepository: UserRepository;
11
12  beforeEach(async () => {
13    const module: TestingModule = await Test.createTestingModule({
14      providers: [UserService, UserRepository, { provide: getRepositoryToken(UserEntity), useFactory: jest.fn }],
15    }).compile();
16
17    userRepository = await module.resolve(UserRepository);
18    service = await module.resolve(UserService);
19  });
20
21  test('CreateUser_ShowdBeTrue', async () => {
22    const userDto = new UserDto();
23    userDto.userName = 'fullName_field_value';
24    userDto.password = 'password_field_value';
25
26    jest.spyOn(userRepository, 'create').mockResolvedValue(new UserEntity());
27
28    const test = await service.createUser(userDto);
29    expect(test).toEqual(userDto.userName);
30  });
31 });
32

```

PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE

```

PASS src/features/user/v1/tests/user.service.spec.ts (10.898 s)
  UserService
    ✓ CreateUser_ShowdBeTrue (15 ms)

```

Test Suites: 1 passed, 1 total

Tests: 1 passed, 1 total

Snapshots: 0 total

Time: 11.014 s

Ran all test suites matching /user.service.spec.ts/i.

PS D:\develop\git\article>

TDD: Test Driven Development

O Desenvolvimento Orientado a Testes, ou TDD, inverte a abordagem convencional de engenharia ao ditar que **o código de teste deve ser escrito antes da implementação do código de produção**. O processo segue um ciclo estrito e repetitivo dividido em três fases conhecido pelo jargão **Red, Green, Refactor**.

Na fase **Red**, o desenvolvedor escreve **um teste unitário para uma funcionalidade que ainda não existe**, observando-o falhar na execução. Na fase **Green**, escreve-se o **código de produção mínimo e estritamente necessário para fazer o teste passar**. Finalmente, na fase **Refactor**, **o código é limpo e aprimorado, mantendo a garantia de que o comportamento continua correto**.

Arquitetura de Camadas

A organização física de arquivos e diretórios em um projeto de backend deve refletir a separação clara de responsabilidades arquiteturais do sistema. Projetos profissionais evitam estruturas desordenadas adotando padrões de arquitetura em camadas, como a **Arquitetura Limpa** ou **Arquitetura Cebola**.

Sob essa ótica, o código é segmentado em pastas específicas para entidades de negócio, casos de uso lógicos, controladores de entrada e adaptadores de infraestrutura externa. Essa padronização permite que desenvolvedores localizem componentes rapidamente, facilita a substituição de tecnologias de banco de dados e impede o acoplamento nocivo entre a lógica de negócio e os frameworks utilizados.

O Padrão SOLID

O acrônimo SOLID representa um conjunto de cinco princípios de design orientado a objetos formalizados por Robert C. Martin no início dos anos 2000. Esses princípios funcionam como diretrizes teóricas destinadas a combater os três principais sintomas de um código degradado: a rigidez para sofrer alterações, a fragilidade que faz o sistema quebrar em pontos inesperados e a imobilidade de reaproveitamento.

Ao aplicar o SOLID, o engenheiro de software transforma estruturas de código altamente acopladas e interdependentes em componentes altamente coesos e modulares, facilitando a expansão do sistema à medida que os requisitos de negócio evoluem.

S – Princípio da Responsabilidade Única

O **Princípio da Responsabilidade Única** estabelece que uma classe deve possuir **apenas uma única razão para mudar**, operando com foco exclusivo em uma única tarefa do sistema. Considere como exemplo negativo uma classe denominada Pedido que contém atributos do produto, calcula o valor total da compra, realiza consultas SQL no banco de dados e gera um arquivo PDF de relatório.

Qualquer alteração no formato do relatório ou na regra de conexão com o banco exigiria a modificação da classe Pedido. A correção consiste em separar essas funções em classes distintas como Pedido, PedidoRepository e RelatorioPedidoService.

O – Princípio do Aberto e Fechado

O Princípio do Aberto e Fechado determina que as entidades de software, como classes, módulos e funções, **devem estar abertas para extensão, mas estritamente fechadas para modificação**. Isso significa que para **adicionar um novo comportamento ao sistema, o desenvolvedor não deve alterar o código-fonte existente que já está testado e funcionando**.

Essa meta é alcançada de forma prática através do uso de **abstrações, interfaces e polimorfismo**. Em vez de uma classe de processamento usar uma estrutura condicional if/else para tratar diferentes tipos de pagamento, ela deve interagir com uma interface genérica MeioPagamento, permitindo a inclusão de novas modalidades sem tocar no motor principal.

L – Princípio da Substituição de Liskov

Formulado por Barbara Liskov, este **princípio dita que os objetos de uma subclasse devem ser capazes de substituir os objetos de uma superclasse** em qualquer parte do programa sem alterar a correção do sistema. O exemplo clássico de violação ocorre ao **modelar uma classe Quadrado herdando de uma classe Retângulo**.

Como o quadrado possui restrições geométricas onde a largura e a altura devem ser idênticas, modificar a largura de uma instância de Quadrado altera implicitamente sua altura, quebrando o comportamento esperado de um retângulo genérico. O princípio exige que as heranças respeitem os contratos comportamentais da classe pai e não apenas a semântica textual.

I – Princípio da Segregação de Interfaces

O Princípio da Segregação de Interfaces preconiza que uma classe cliente **nunca deve ser forçada** a depender de métodos de uma interface que **ela própria não utiliza**. Na prática, este conceito orienta a criação de **múltiplas interfaces específicas e enxutas** em detrimento de uma única interface genérica e inflada de métodos conhecidos como "interfaces gordas".

Se uma interface `DispositivoMultiFuncao` declara os métodos `imprimir()`, `copiar()` e `enviarFax()`, uma classe `ImpressoraSimples` que a implemente será obrigada a deixar o método de fax vazio ou lançando exceções de erro, violando o design. **A solução reside em fracionar a estrutura em interfaces menores.**

D – Princípio da Inversão de Dependência

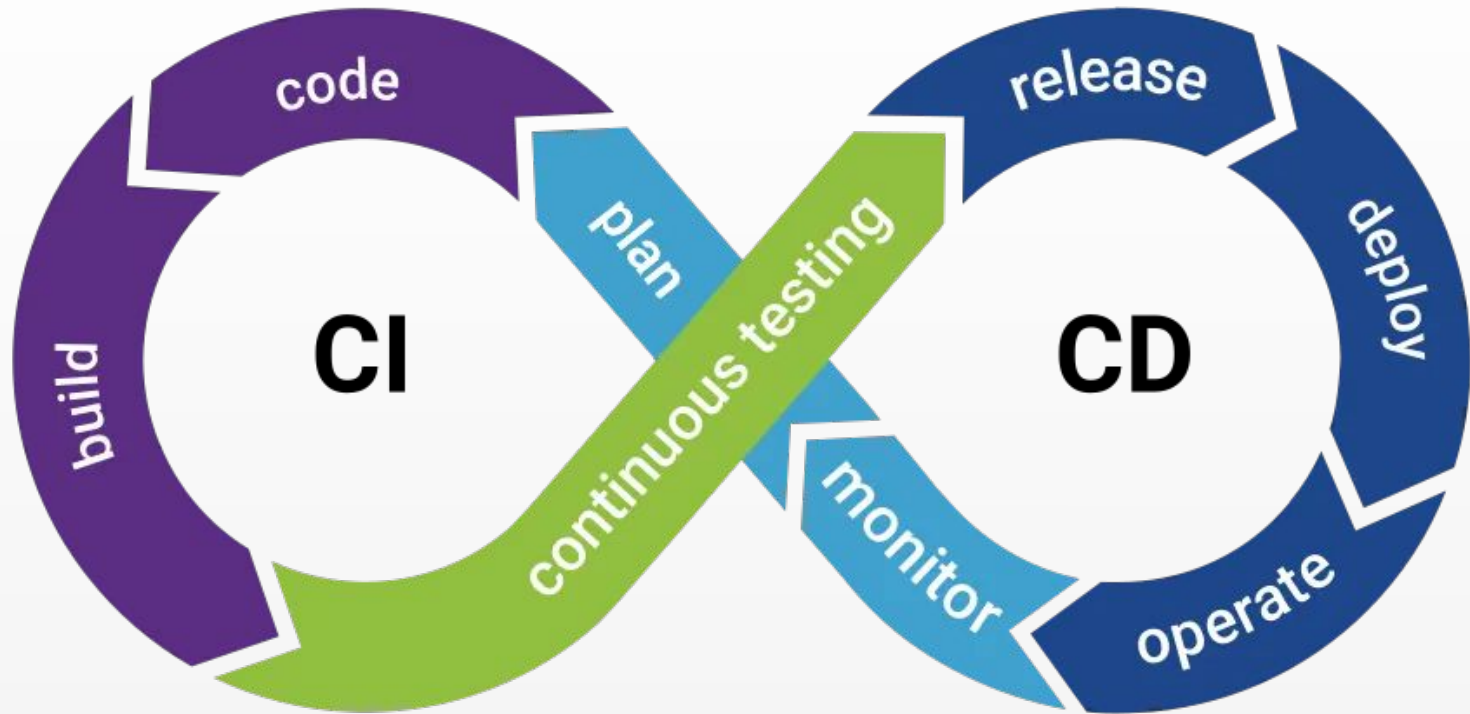
O último pilar do SOLID dita que os **módulos de alto nível, que contêm as regras de negócio essenciais, não devem depender de módulos de baixo nível voltados a detalhes de infraestrutura**; ambos devem depender de abstrações. Além disso, as abstrações não devem depender de detalhes, mas os **detalhes devem depender de abstrações**.

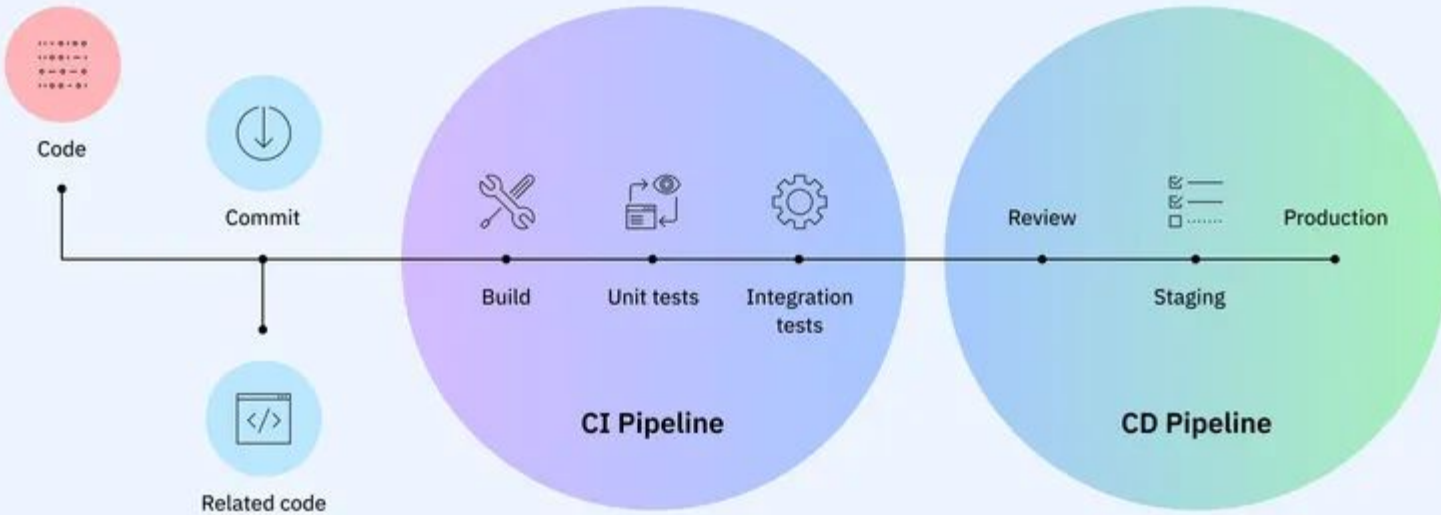
Na prática, uma classe de serviço de backend não deve instanciar diretamente uma classe de banco de dados específica como MySqlConnection utilizando a palavra-chave new. Ela deve receber através de seu construtor uma interface genérica DatabaseConnection, invertendo o fluxo de acoplamento do sistema.

Automação e Integração Contínua (CI)

A existência de testes unitários perde eficácia prática se a execução dos testes depender exclusivamente da iniciativa manual de cada programador antes do envio do código. Em ambientes corporativos de engenharia de software, o repositório Git é conectado a servidores de **Integração Contínua (CI)**, como o **GitHub Actions** ou **GitLab CI**.

Toda vez que um desenvolvedor envia um commit ou solicita a fusão de uma ramificação, **o servidor de CI intercepta o evento, monta o ambiente virtual de backend automaticamente e executa toda a suíte de testes unitários**. Se algum teste falhar, a **fusão do código é bloqueada**, impedindo a contaminação do ambiente de produção.





Abordagem Estruturada

A aplicação rigorosa de testes, versionamento consciente e princípios SOLID gera um retorno sobre o **investimento imenso ao longo do ciclo de vida útil do software**. O código torna-se **resiliente a refatorações**, permitindo que a equipe limpe e otimize o sistema com a segurança de que os testes automatizados acusarão qualquer quebra de comportamento.

O histórico do Git atua como uma máquina do tempo precisa **para auditorias de segurança**, enquanto a modularidade derivada do SOLID reduz expressivamente o tempo de integração de novos membros na equipe de desenvolvimento, uma vez que as responsabilidades do sistema estão claramente isoladas.

Questionário



Links repositórios:

<https://github.com/FabricaDeSinapse/nestjs-documentacao-swagger/blob/main/README.md>

<https://consolelog.com.br/nestjs-swagger-automatizar-geracao-documentacao-api/>

<https://medium.com/@rcalmeida/nestjs-jest-typeorm-criando-testes-unit%C3%A1rios-4731ad780b7b>

<https://www.luiztools.com.br/post/como-testar-controllers-em-nestjs-com-jest/>

<https://www.luiztools.com.br/post/como-testar-services-em-nestjs-com-jest/>

<https://dev.to/renatoadorno/padroes-de-commits-commit-patterns-4lco>

<https://github.com/iuricode/padroes-de-commits>